Modelos SARIMA en Series de Tiempo Financieras

Teoría, Identificación, Estimación y Aplicaciones Prácticas con Python

Notas de curso: Series de Tiempo Aplicadas a Finanzas
Universidad Panamericana — Ingeniería Financiera

Abril 2026

Contents

1 Introducción **3**
1.1 Relevancia en Finanzas . . . 3
1.2 Limitaciones y contexto . . . 3

2 Forma General del Modelo SARIMA **3**
2.1 Notación y operadores . . . 3
2.2 Definición formal del modelo SARIMA$(p, d, q)(P, D, Q)_s$ . . . 4
2.3 Descripción de parámetros . . . 4
2.3.1 Componentes no estacionales . . . 4
2.3.2 Componentes estacionales . . . 5
2.4 Condiciones de estacionariedad e invertibilidad . . . 5

3 Herramientas de Identificación **5**
3.1 Función de Autocorrelación (ACF) . . . 5
3.2 Función de Autocorrelación Parcial (PACF) . . . 6
3.3 Criterio de Información de Akaike (AIC) . . . 6
3.4 Criterio de Información Bayesiano (BIC) . . . 7
3.5 Diferenciación: d y D . . . 7
3.6 Estacionariedad . . . 7
3.7 Ruido Blanco . . . 7

4 Metodología Box-Jenkins **7**
4.1 Etapa 1: Identificación . . . 7
4.2 Etapa 2: Estimación . . . 8
4.3 Etapa 3: Validación . . . 8
4.4 Etapa 4: Diagnóstico de residuos . . . 8

5 Ejemplos Prácticos con Python **8**
5.1 Ejemplo 1: Tipo de cambio USD/MXN . . . 8
5.1.1 Descripción de los datos . . . 8
5.1.2 Código Python . . . 9
5.1.3 Interpretación . . . 11
5.2 Ejemplo 2: Retornos mensuales del S&P 500 . . . 11

5.2.1 Descripción de los datos 11
5.2.2 Código Python 11
5.2.3 Interpretación 13
5.3 Ejemplo 3: Petróleo WTI — SARIMA con diferenciación estacional . 13
5.3.1 Descripción de los datos 13
5.3.2 Código Python 13
5.3.3 Interpretación 16

6 Conclusiones **17**
6.1 Síntesis metodológica 17
6.2 Aplicaciones en finanzas reales 17
6.3 Limitaciones y extensiones 17

Referencias **17**

1 Introducción

El análisis de series de tiempo es uno de los pilares fundamentales de las finanzas cuantitativas. Desde la predicción de precios de activos y tipos de cambio hasta la modelación de volatilidad y riesgo de crédito, los modelos paramétricos de series de tiempo permiten extraer estructura estadística de datos secuenciales y generar pronósticos probabilísticos rigurosos.

Dentro de esta familia de modelos, los modelos **ARIMA** (*Autoregressive Integrated Moving Average*) propuestos formalmente por Box y Jenkins (1970) constituyen la base de la econometría de series de tiempo univariadas. Su extensión natural, los modelos **SARIMA** (*Seasonal ARIMA*), incorpora patrones estacionales deterministas presentes en muchas series financieras: rendimientos mensuales de fondos de inversión, tasas de interés con ciclos anuales, volúmenes de negociación con patrones semanales, entre otros.

1.1 Relevancia en Finanzas

Las aplicaciones de los modelos SARIMA en finanzas son amplias:

- **Tipos de cambio**: Las series de FX presentan con frecuencia autocorrelaciones de corto plazo y ocasionalmente patrones estacionales asociados a flujos comerciales.
- **Tasas de interés**: Las tasas de política monetaria y curvas de rendimiento exhiben alta persistencia y ciclos asociados a decisiones de bancos centrales.
- **Volumen bursátil**: Los volúmenes de negociación presentan estacionalidad intradiaria, semanal y mensual (días de vencimiento de opciones, rebalanceos de índices).
- **Commodities**: Los precios de energía y agrícolas presentan estacionalidad climática y de demanda.
- **Riesgo de crédito**: Los índices de morosidad pueden mostrar patrones estacionales relacionados con ciclos económicos.

1.2 Limitaciones y contexto

Es importante señalar que los modelos SARIMA pertenecen a la clase de modelos lineales para la media condicional de la serie. No modelan directamente la volatilidad condicional (para ello existen modelos GARCH), ni capturan no linealidades complejas (modelos de redes neuronales, SETAR, etc.). Sin embargo, su interpretabilidad y parsimonia los hacen herramientas de referencia obligatoria en cualquier análisis de series de tiempo.

2 Forma General del Modelo SARIMA

2.1 Notación y operadores

Denotamos la serie de tiempo como $\{y_t\}_{t=1}^{T}$, donde t indexa el tiempo. Introducimos dos operadores esenciales:

Definición 2.1 (Operador de rezago). *El* ***operador de rezago*** B *se define como:*

$$\mathsf{B}^k y_t = y_{t-k}, \quad k \in \mathbb{Z}_{\geq 0}.$$

Definición 2.2 (Operador de diferencia ordinaria). *El* ***operador de diferencia*** *de orden d se define como:*

$$\nabla^d y_t = (1 - \mathsf{B})^d y_t.$$

En particular, $\nabla y_t = y_t - y_{t-1}$ (diferencia de primer orden).

Definición 2.3 (Operador de diferencia estacional). *El* ***operador de diferencia estacional*** *de orden D con periodo s se define como:*

$$\nabla_s^D y_t = (1 - \mathsf{B}^s)^D y_t.$$

Para $D = 1$: $\nabla_s y_t = y_t - y_{t-s}$.

2.2 Definición formal del modelo SARIMA$(p, d, q)(P, D, Q)_s$

Definición 2.4 (Modelo SARIMA). *Un proceso $\{y_t\}$ sigue un modelo SARIMA$(p, d, q)(P, D, Q)_s$ si satisface la ecuación:*

$$\boxed{\Phi_P(\mathsf{B}^s)\,\phi_p(\mathsf{B})\,(1-\mathsf{B})^d(1-\mathsf{B}^s)^D\,y_t = \Theta_Q(\mathsf{B}^s)\,\theta_q(\mathsf{B})\,\varepsilon_t} \tag{1}$$

donde $\varepsilon_t \sim RB(0, \sigma^2)$ es ruido blanco con varianza σ^2.

Polinomios no estacionales:

$$\phi_p(\mathsf{B}) = 1 - \phi_1\mathsf{B} - \phi_2\mathsf{B}^2 - \cdots - \phi_p\mathsf{B}^p, \tag{2}$$
$$\theta_q(\mathsf{B}) = 1 + \theta_1\mathsf{B} + \theta_2\mathsf{B}^2 + \cdots + \theta_q\mathsf{B}^q. \tag{3}$$

Polinomios estacionales:

$$\Phi_P(\mathsf{B}^s) = 1 - \Phi_1\mathsf{B}^s - \Phi_2\mathsf{B}^{2s} - \cdots - \Phi_P\mathsf{B}^{Ps}, \tag{4}$$
$$\Theta_Q(\mathsf{B}^s) = 1 + \Theta_1\mathsf{B}^s + \Theta_2\mathsf{B}^{2s} + \cdots + \Theta_Q\mathsf{B}^{Qs}. \tag{5}$$

2.3 Descripción de parámetros

2.3.1 Componentes no estacionales

p — **Orden autorregresivo**
Indica cuántos rezagos de la serie (ya diferenciada) entran como predictores lineales. Un AR(p) captura la inercia o memoria de corto plazo del proceso. *Interpretación financiera*: $p = 1$ en rendimientos implica que el rendimiento de hoy está parcialmente determinado por el de ayer (posible autocorrelación de primer orden, frecuente en retornos diarios de activos ilíquidos o con efectos momentum débiles).

d — **Orden de integración ordinaria**
Número de diferencias ordinarias necesarias para que la serie sea estacionaria. En series de precios de activos financieros se tiene típicamente $d = 1$, pues los precios son no estacionarios (caminata aleatoria) pero los rendimientos logarítmicos son estacionarios.

q — **Orden de media móvil**
Número de shocks pasados (errores de pronóstico) que afectan el nivel actual de la serie. Un MA(q) implica que el proceso tiene *memoria finita* de exactamente q periodos.

2.3.2 Componentes estacionales

P — **Orden autorregresivo estacional**
Número de rezagos estacionales de la serie diferenciada que actúan como predictores. Captura la dependencia entre observaciones separadas exactamente s periodos.

D — **Orden de integración estacional**
Número de diferencias estacionales necesarias para eliminar tendencias estacionales. Una diferencia estacional $D = 1$ sustrae el valor del mismo periodo del año anterior: $y_t - y_{t-s}$.

Q — **Orden de media móvil estacional**
Número de shocks estacionales pasados incluidos en el modelo.

s — **Periodo estacional**
Frecuencia del ciclo estacional. Valores comunes en finanzas:

- $s = 4$: datos trimestrales con ciclo anual.
- $s = 12$: datos mensuales con ciclo anual.
- $s = 5$ o $s = 22$: datos diarios con ciclo semanal o mensual.
- $s = 252$: datos diarios con ciclo anual (días hábiles).

2.4 Condiciones de estacionariedad e invertibilidad

Para que el modelo SARIMA sea estacionario e invertible, se requiere que todas las raíces de los polinomios $\phi_p(z) = 0$ y $\Phi_P(z^s) = 0$ estén fuera del círculo unitario en el plano complejo, y análogamente para $\theta_q(z) = 0$ y $\Theta_Q(z^s) = 0$.

3 Herramientas de Identificación

3.1 Función de Autocorrelación (ACF)

Definición 3.1 (ACF). *Para un proceso estacionario $\{y_t\}$ con media μ y varianza $\sigma_y^2 < \infty$, la **función de autocorrelación** (ACF) en el rezago k se define como:*

$$\rho(k) = \frac{\mathrm{Cov}(y_t, y_{t-k})}{\mathrm{Var}(y_t)} = \frac{\gamma(k)}{\gamma(0)}, \quad k = 0, \pm 1, \pm 2, \ldots$$

donde $\gamma(k) = \mathrm{Cov}(y_t, y_{t-k})$ es la función de autocovarianza.

Estimador muestral:

$$\hat{\rho}(k) = \frac{\sum_{t=k+1}^{T} (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^{T} (y_t - \bar{y})^2}.$$

Uso en selección de modelos.

- Un proceso $\mathbf{MA}(q)$ presenta ACF que se *trunca* exactamente en el rezago q: $\rho(k) = 0$ para $|k| > q$.
- Un proceso $\mathbf{AR}(p)$ presenta ACF que *decae* exponencialmente.
- Picos significativos en rezagos $k = s, 2s, 3s, \ldots$ indican componentes estacionales.
- Bandas de confianza aproximadas al 95%: $\pm 1.96/\sqrt{T}$.

3.2 Función de Autocorrelación Parcial (PACF)

Definición 3.2 (PACF). *La **función de autocorrelación parcial** (PACF) en el rezago k, denotada $\alpha(k)$, mide la correlación entre y_t e y_{t-k}* eliminando *el efecto lineal de las variables intermedias $y_{t-1}, y_{t-2}, \ldots, y_{t-k+1}$:*

$$\alpha(k) = \mathrm{Corr}(y_t - \hat{y}_t^{(k)},\ y_{t-k} - \hat{y}_{t-k}^{(k)}),$$

donde $\hat{y}_t^{(k)}$ es la proyección lineal de y_t sobre $\{y_{t-1}, \ldots, y_{t-k+1}\}$.

Uso en selección de modelos.

- Un proceso $\mathbf{AR}(p)$ presenta PACF que se *trunca* exactamente en el rezago p.
- Un proceso $\mathbf{MA}(q)$ presenta PACF que *decae* gradualmente.

Table 1: Patrones de ACF y PACF según el proceso generador de datos

Proceso	**ACF**	**PACF**
AR(p)	Decae exponencialmente	Se trunca en rezago p
MA(q)	Se trunca en rezago q	Decae gradualmente
ARMA(p, q)	Decae gradualmente	Decae gradualmente
Ruido blanco	≈ 0 para $k \geq 1$	≈ 0 para $k \geq 1$

3.3 Criterio de Información de Akaike (AIC)

Definición 3.3 (AIC). *Sea $\hat{L}$ el valor máximo de la función de verosimilitud del modelo con k parámetros libres. El **criterio de Akaike** se define como:*

$$\mathrm{AIC} = -2\ln(\hat{L}) + 2k.$$

El AIC penaliza la complejidad del modelo frente al ajuste. Se prefiere el modelo con **menor AIC** al comparar especificaciones alternativas.

3.4 Criterio de Información Bayesiano (BIC)

Definición 3.4 (BIC)**.**

$$\mathrm{BIC} = -2\ln(\hat{L}) + k\ln(T),$$

donde T es el tamaño de la muestra.

El BIC penaliza más severamente la complejidad que el AIC (pues $\ln(T) > 2$ para $T > 7$), y es *consistente*: cuando $T \to \infty$, selecciona el verdadero modelo generador de datos si éste pertenece a la familia considerada.

3.5 Diferenciación: d y D

Las pruebas formales para determinar d y D incluyen:

- **Prueba de Dickey-Fuller Aumentada (ADF)**: contrasta H_0: raíz unitaria vs. H_1: estacionariedad.
- **Prueba KPSS**: contrasta H_0: estacionariedad vs. H_1: raíz unitaria.
- **Criterio visual**: gráfica de la serie y correlograma (ACF muy persistente sugiere no estacionariedad).

3.6 Estacionariedad

Definición 3.5 (Estacionariedad débil)**.** *Un proceso $\{y_t\}$ es **débilmente estacionario** si:*

1. $\mathbb{E}[y_t] = \mu < \infty$ *para todo t,*
2. $\mathrm{Var}(y_t) = \sigma^2 < \infty$ *para todo t,*
3. $\mathrm{Cov}(y_t, y_{t-k}) = \gamma(k)$ *depende solo de k, no de t.*

Los precios de activos son típicamente $I(1)$ (no estacionarios), mientras que los rendimientos logarítmicos $r_t = \ln(P_t/P_{t-1})$ suelen ser estacionarios $I(0)$.

3.7 Ruido Blanco

Definición 3.6 (Ruido blanco)**.** *Un proceso $\{\varepsilon_t\}$ es **ruido blanco** si:*

$$\mathbb{E}[\varepsilon_t] = 0, \quad \mathbb{E}[\varepsilon_t^2] = \sigma^2, \quad \mathbb{E}[\varepsilon_t \varepsilon_s] = 0 \text{ para } t \neq s.$$

Los residuos de un modelo SARIMA bien especificado deben comportarse como ruido blanco, verificado mediante la **prueba de Ljung-Box**:

$$Q_{LB} = T(T+2)\sum_{k=1}^{m} \frac{\hat{\rho}_\varepsilon^2(k)}{T-k} \overset{H_0}{\sim} \chi^2(m-p-q-P-Q).$$

4 Metodología Box-Jenkins

4.1 Etapa 1: Identificación

1. **Graficar la serie**: detectar visualmente tendencias, quiebres estructurales, heterocedasticidad y patrones estacionales.

2. **Transformaciones**: si la varianza es no constante, aplicar la transformación logarítmica $\tilde{y}_t = \ln(y_t)$ o la transformación Box-Cox.
3. **Diferenciación**: determinar d y D mediante pruebas de raíz unitaria.
4. **Análisis ACF y PACF**: examinar correlogramas para proponer órdenes tentativos.
5. **Definir candidatos**: listar un conjunto de modelos SARIMA de bajo orden.

4.2 Etapa 2: Estimación

Los parámetros del modelo se estiman por **máxima verosimilitud condicional** (MLE). Bajo normalidad de ε_t, la log-verosimilitud condicional es:

$$\ell(\boldsymbol{\phi}, \boldsymbol{\theta}, \boldsymbol{\Phi}, \boldsymbol{\Theta}, \sigma^2 \mid \mathbf{y}) = -\frac{T^*}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_t \varepsilon_t^2,$$

donde $T^* = T - d - Ds$ es el número efectivo de observaciones.

4.3 Etapa 3: Validación

1. Comparar AIC y BIC: seleccionar el modelo de menor valor.
2. Verificar significancia estadística de los coeficientes estimados.
3. Comprobar invertibilidad y estacionariedad (raíces fuera del círculo unitario).
4. Principio de parsimonia: ante modelos similares, preferir el más simple.

4.4 Etapa 4: Diagnóstico de residuos

1. **Gráfica de residuos**: ausencia de patrones o heterocedasticidad.
2. **ACF/PACF de residuos**: picos dentro de las bandas de confianza.
3. **Prueba de Ljung-Box**: p-valor > 0.05.
4. **Normalidad**: histograma y QQ-plot; prueba de Jarque-Bera.
5. Si el diagnóstico falla, regresar a la Etapa 1.

5 Ejemplos Prácticos con Python

Los tres ejemplos siguientes utilizan datos financieros reales descargados mediante `yfinance`. El paquete `statsmodels` provee la implementación de SARIMA a través de `SARIMAX`.

5.1 Ejemplo 1: Tipo de cambio USD/MXN

5.1.1 Descripción de los datos

El tipo de cambio peso mexicano frente al dólar estadounidense (USD/MXN) es una serie diaria con alta persistencia. Los precios del FX son $I(1)$: la serie de niveles tiene raíz unitaria, pero los rendimientos logarítmicos diarios son estacionarios.

5.1.2 Código Python

```python
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.stats.diagnostic import acorr_ljungbox
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# 1. Descarga de datos
ticker = yf.Ticker("MXN=X")
df = ticker.history(period="2y", interval="1d")
precio = df['Close'].dropna()
rendimiento = np.log(precio).diff().dropna()

# 2. Visualizacion
fig, axes = plt.subplots(2, 1, figsize=(12, 6))
axes[0].plot(precio, color='steelblue', linewidth=0.8)
axes[0].set_title('Precio USD/MXN (cierre diario)')
axes[1].plot(rendimiento, color='darkred', linewidth=0.6)
axes[1].axhline(0, color='black', linewidth=0.5)
axes[1].set_title('Rendimiento logaritmico diario')
plt.tight_layout()
plt.savefig('usdmxn_serie.png', dpi=150)
plt.show()

# 3. Prueba ADF
def adf_test(serie, nombre):
    res = adfuller(serie, autolag='AIC')
    print(f"ADF {nombre}: stat={res[0]:.4f}, p={res[1]:.4f}")
    decision = "Estacionaria" if res[1] < 0.05 else "NO
        estacionaria"
    print(f"  -> {decision}")

adf_test(precio, "Precio USD/MXN")
adf_test(rendimiento, "Rendimiento log USD/MXN")

# 4. ACF y PACF
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(rendimiento, lags=30, ax=axes[0],
         title='ACF - Rendimiento USD/MXN')
plot_pacf(rendimiento, lags=30, ax=axes[1],
          title='PACF - Rendimiento USD/MXN')
plt.tight_layout()
plt.savefig('usdmxn_acf_pacf.png', dpi=150)
plt.show()

# 5. Grid search AIC
resultados = []
for p in range(0, 4):
    for q in range(0, 4):
        try:
            m = SARIMAX(rendimiento, order=(p, 0, q),
```

```
                        trend='n', enforce_stationarity=True,
                        enforce_invertibility=True)
            r = m.fit(disp=False)
            resultados.append({'p': p, 'q': q,
                               'AIC': r.aic, 'BIC': r.bic})
        except Exception:
            pass

df_res = pd.DataFrame(resultados).sort_values('AIC')
print("\nTop 5 modelos por AIC:")
print(df_res.head())

# 6. Modelo optimo
fila = df_res.iloc[0]
p_opt, q_opt = int(fila['p']), int(fila['q'])
print(f"\nModelo: ARIMA({p_opt}, 0, {q_opt})")

modelo_final = SARIMAX(rendimiento,
                       order=(p_opt, 0, q_opt),
                       trend='n').fit(disp=False)
print(modelo_final.summary())

# 7. Diagnostico de residuos
residuos = modelo_final.resid
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes[0, 0].plot(residuos, color='navy', linewidth=0.6)
axes[0, 0].set_title('Residuos')
plot_acf(residuos, lags=20, ax=axes[0, 1],
         title='ACF Residuos')
axes[1, 0].hist(residuos, bins=40, color='steelblue',
                edgecolor='white')
axes[1, 0].set_title('Histograma de Residuos')
stats.probplot(residuos, dist='norm', plot=axes[1, 1])
axes[1, 1].set_title('QQ-Plot')
plt.tight_layout()
plt.savefig('usdmxn_diagnostico.png', dpi=150)
plt.show()

lb = acorr_ljungbox(residuos, lags=[10, 20],
                    return_df=True)
print("\nLjung-Box:")
print(lb)

# 8. Prediccion 10 dias
pred = modelo_final.get_forecast(steps=10)
pred_ci = pred.conf_int(alpha=0.05)
print("\nPrediccion 10 dias (rendimiento log):")
print(pd.DataFrame({
    'Pred': pred.predicted_mean,
    'IC_inf': pred_ci.iloc[:, 0],
    'IC_sup': pred_ci.iloc[:, 1]
}))
```

Listing 1: Ejemplo 1: ARIMA para rendimientos USD/MXN

5.1.3 Interpretación

La prueba ADF rechaza la raíz unitaria en los rendimientos logarítmicos del USD/MXN, confirmando la estacionariedad. El análisis ACF/PACF muestra autocorrelaciones pequeñas pero significativas en algunos rezagos, lo que motiva la inclusión de términos AR y/o MA de orden bajo. El grid search selecciona el modelo con menor AIC, típicamente ARIMA(1,0,1) o ARIMA(0,0,1) para rendimientos de FX. La prueba de Ljung-Box no debe rechazar H_0 de no autocorrelación en los residuos.

5.2 Ejemplo 2: Retornos mensuales del S&P 500

5.2.1 Descripción de los datos

El índice S&P 500 en frecuencia mensual puede exhibir patrones estacionales relacionados con ciclos de ganancias corporativas, rebalanceos institucionales y efectos de calendario. Utilizaremos 10 años de datos mensuales con $s = 12$ (ciclo anual).

5.2.2 Código Python

```python
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.stats.diagnostic import acorr_ljungbox
import warnings
warnings.filterwarnings('ignore')

# 1. Descarga y resampleo mensual
sp500 = yf.download("^GSPC", period="10y",
                    interval="1mo", progress=False)
precio_m = sp500['Close'].dropna()
retorno_m = np.log(precio_m).diff().dropna() * 100  # en pct

print(f"Observaciones: {len(retorno_m)}")
print(retorno_m.describe())

# 2. Visualizacion
fig, axes = plt.subplots(2, 1, figsize=(12, 6))
axes[0].plot(precio_m, color='steelblue')
axes[0].set_title('S&P 500 (precio mensual de cierre)')
colors = ['green' if r >= 0 else 'red' for r in retorno_m]
axes[1].bar(retorno_m.index, retorno_m, color=colors,
            width=20)
axes[1].set_title('Retorno logaritmico mensual (%)')
plt.tight_layout()
plt.savefig('sp500_serie.png', dpi=150)
plt.show()

# 3. Prueba ADF
r_adf = adfuller(retorno_m, autolag='AIC')
print(f"\nADF retornos: stat={r_adf[0]:.4f}, "
      f"p={r_adf[1]:.4f}")

```

```python
# 4. ACF/PACF hasta rezago 36
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(retorno_m, lags=36, ax=axes[0],
         title='ACF - Retorno mensual S&P 500')
plot_pacf(retorno_m, lags=36, ax=axes[1],
          title='PACF - Retorno mensual S&P 500')
plt.tight_layout()
plt.savefig('sp500_acf_pacf.png', dpi=150)
plt.show()

# 5. Grid search SARIMA(p,0,q)(P,0,Q)_12
S = 12
resultados = []
for p in range(0, 3):
    for q in range(0, 3):
        for P in range(0, 2):
            for Q in range(0, 2):
                try:
                    m = SARIMAX(retorno_m,
                                order=(p, 0, q),
                                seasonal_order=(P, 0, Q, S),
                                trend='c')
                    r = m.fit(disp=False, maxiter=200)
                    resultados.append({
                        'p': p, 'q': q, 'P': P, 'Q': Q,
                        'AIC': r.aic, 'BIC': r.bic
                    })
                except Exception:
                    pass

df_grid = pd.DataFrame(resultados).sort_values('AIC')
print("\nTop 5 modelos SARIMA por AIC:")
print(df_grid.head())

# 6. Ajuste del modelo optimo
f = df_grid.iloc[0]
p_o, q_o = int(f.p), int(f.q)
P_o, Q_o = int(f.P), int(f.Q)
print(f"\nModelo: SARIMA({p_o},0,{q_o})({P_o},0,{Q_o})_12")

modelo = SARIMAX(retorno_m, order=(p_o, 0, q_o),
                 seasonal_order=(P_o, 0, Q_o, S),
                 trend='c').fit(disp=False, maxiter=300)
print(modelo.summary())

# 7. Diagnostico
modelo.plot_diagnostics(figsize=(12, 8))
plt.suptitle('Diagnostico - S&P 500 SARIMA', y=1.02)
plt.tight_layout()
plt.savefig('sp500_diagnostico.png', dpi=150)
plt.show()

lb = acorr_ljungbox(modelo.resid, lags=[12, 24],
                    return_df=True)
print("\nLjung-Box:")
print(lb)

# 8. Prediccion 12 meses
```

```
fc = modelo.get_forecast(steps=12)
fc_mean = fc.predicted_mean
fc_ci = fc.conf_int(alpha=0.05)

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(retorno_m[-36:], label='Historico',
        color='steelblue')
ax.plot(fc_mean, label='Prediccion',
        color='darkorange', linestyle='--')
ax.fill_between(fc_ci.index,
                fc_ci.iloc[:, 0], fc_ci.iloc[:, 1],
                alpha=0.25, color='orange', label='IC 95%')
ax.axhline(0, color='black', linewidth=0.5)
ax.set_title('Prediccion 12 meses - Retorno S&P 500')
ax.legend()
plt.tight_layout()
plt.savefig('sp500_forecast.png', dpi=150)
plt.show()
```

Listing 2: Ejemplo 2: SARIMA para S&P 500 mensual

5.2.3 Interpretación

Los retornos logarítmicos mensuales del S&P 500 son estacionarios con media positiva pequeña (prima de riesgo) y desviación estándar elevada. La ACF mensual puede mostrar picos en rezagos múltiplos de 12, lo que justifica la componente estacional con $s = 12$. El BIC típicamente selecciona modelos parsimoniosos como $\text{SARIMA}(1,0,0)(0,0,1)_{12}$. Los intervalos de pronóstico amplios reflejan la alta incertidumbre inherente a los retornos bursátiles.

5.3 Ejemplo 3: Petróleo WTI — SARIMA con diferenciación estacional

5.3.1 Descripción de los datos

Los precios del petróleo crudo WTI exhiben fuerte estacionalidad relacionada con la demanda de combustible y los ciclos de producción de la OPEP+. Trabajaremos con datos semanales de los últimos 5 años, aplicando $d = 1$ y $D = 1$ con $s = 52$ (ciclo anual en semanas).

5.3.2 Código Python

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.stats.diagnostic import acorr_ljungbox
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

```

```python
# 1. Descarga datos WTI semanales
wti = yf.download("CL=F", period="5y",
                  interval="1wk", progress=False)
precio_wti = wti['Close'].dropna()
log_precio = np.log(precio_wti)

print(f"Observaciones: {len(precio_wti)}")
print(precio_wti.describe())

# 2. Visualizacion
fig, axes = plt.subplots(3, 1, figsize=(12, 9))
axes[0].plot(precio_wti, color='saddlebrown', lw=0.8)
axes[0].set_title('Precio WTI (cierre semanal, USD/barril)')
axes[1].plot(log_precio, color='darkorange', lw=0.8)
axes[1].set_title('Log-precio WTI')
diff1 = log_precio.diff().dropna()
axes[2].plot(diff1, color='navy', lw=0.6)
axes[2].axhline(0, color='black', lw=0.5)
axes[2].set_title('Primera diferencia del log-precio WTI')
plt.tight_layout()
plt.savefig('wti_serie.png', dpi=150)
plt.show()

# 3. Pruebas de estacionariedad
def adf_kpss(serie, nombre):
    adf_r = adfuller(serie, autolag='AIC')
    kpss_r = kpss(serie, regression='c', nlags='auto')
    print(f"\n--- {nombre} ---")
    print(f"  ADF  stat={adf_r[0]:.4f}, p={adf_r[1]:.4f}")
    print(f"  KPSS stat={kpss_r[0]:.4f}, p={kpss_r[1]:.4f}")

adf_kpss(log_precio, "Log-precio WTI (nivel)")
adf_kpss(diff1, "Diferencia log-precio WTI")

# 4. ACF y PACF
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(diff1, lags=60, ax=axes[0],
         title='ACF - Diff Log-precio WTI')
plot_pacf(diff1, lags=60, ax=axes[1],
          title='PACF - Diff Log-precio WTI')
plt.tight_layout()
plt.savefig('wti_acf_pacf.png', dpi=150)
plt.show()

# 5. Diferenciacion estacional (D=1, s=52)
diff_est = log_precio.diff(52).diff(1).dropna()
adf_kpss(diff_est, "Diff estacional + ordinaria")

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(diff_est, lags=60, ax=axes[0],
         title='ACF - Diff estacional WTI')
plot_pacf(diff_est, lags=60, ax=axes[1],
          title='PACF - Diff estacional WTI')
plt.tight_layout()
plt.savefig('wti_acf_pacf_diff.png', dpi=150)
plt.show()

# 6. Modelos candidatos SARIMA
```

```python
S_wti = 52
candidatos = [
    (1, 1, 0, 0, 1, 0),
    (0, 1, 1, 0, 1, 1),
    (1, 1, 1, 0, 1, 0),
    (1, 1, 1, 0, 1, 1),
    (2, 1, 1, 0, 1, 0),
    (0, 1, 1, 1, 1, 0),
    (1, 1, 0, 1, 1, 1),
]

resultados_wti = []
for (p, d, q, P, D, Q) in candidatos:
    try:
        m = SARIMAX(log_precio,
                    order=(p, d, q),
                    seasonal_order=(P, D, Q, S_wti),
                    trend='n')
        r = m.fit(disp=False, maxiter=300)
        resultados_wti.append({
            'Orden': f"({p},{d},{q})({P},{D},{Q})_{S_wti}",
            'AIC': round(r.aic, 2),
            'BIC': round(r.bic, 2)
        })
        print(f"  SARIMA({p},{d},{q})({P},{D},{Q})_{S_wti} "
              f"AIC={r.aic:.2f} BIC={r.bic:.2f}")
    except Exception as e:
        print(f"  Error: {e}")

df_wti = pd.DataFrame(resultados_wti).sort_values('AIC')
print("\nRanking por AIC:")
print(df_wti)

# 7. Modelo final: SARIMA(1,1,1)(0,1,1)_52
modelo_wti = SARIMAX(log_precio,
                     order=(1, 1, 1),
                     seasonal_order=(0, 1, 1, 52),
                     trend='n').fit(disp=False, maxiter=400)
print(modelo_wti.summary())

# 8. Diagnostico
resid_wti = modelo_wti.resid
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes[0, 0].plot(resid_wti, lw=0.6, color='navy')
axes[0, 0].set_title('Residuos')
plot_acf(resid_wti, lags=30, ax=axes[0, 1],
         title='ACF Residuos')
axes[1, 0].hist(resid_wti, bins=40,
                color='steelblue', edgecolor='white')
axes[1, 0].set_title('Histograma')
stats.probplot(resid_wti, dist='norm', plot=axes[1, 1])
plt.tight_layout()
plt.savefig('wti_diagnostico.png', dpi=150)
plt.show()

lb_wti = acorr_ljungbox(resid_wti,
                        lags=[10, 20, 52],
                        return_df=True)
```

```python
print("\nLjung-Box (WTI):")
print(lb_wti)

# 9. Prediccion 8 semanas
n_steps = 8
fc_wti = modelo_wti.get_forecast(steps=n_steps)
fc_log = fc_wti.predicted_mean
fc_ci_log = fc_wti.conf_int(alpha=0.05)

# Revertir transformacion logaritmica
fc_precio = np.exp(fc_log)
fc_ci_precio = np.exp(fc_ci_log)

print("\nPrediccion WTI (USD/barril) - 8 semanas:")
print(pd.DataFrame({
    'Pred': fc_precio.round(2),
    'IC_inf': fc_ci_precio.iloc[:, 0].round(2),
    'IC_sup': fc_ci_precio.iloc[:, 1].round(2)
}))

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(precio_wti[-52:], color='saddlebrown',
        label='Precio historico (1 ano)')
ax.plot(fc_precio, 'o--', color='darkorange',
        label='Prediccion 8 semanas')
ax.fill_between(fc_ci_precio.index,
                fc_ci_precio.iloc[:, 0],
                fc_ci_precio.iloc[:, 1],
                alpha=0.3, color='orange',
                label='IC 95%')
ax.set_title(
    'Prediccion precio WTI - SARIMA(1,1,1)(0,1,1)_52')
ax.set_ylabel('USD por barril')
ax.legend()
plt.tight_layout()
plt.savefig('wti_forecast.png', dpi=150)
plt.show()
```

Listing 3: Ejemplo 3: SARIMA para petróleo WTI semanal

5.3.3 Interpretación

Los precios del petróleo WTI en niveles son no estacionarios ($I(1)$): la ADF no rechaza la raíz unitaria en log-precios, pero sí en la primera diferencia. Al aplicar también la diferencia estacional de orden 52, se elimina la autocorrelación estacional visible en los picos de la ACF. El modelo SARIMA$(1,1,1)(0,1,1)_{52}$ es equivalente al clásico modelo "airline" estacional, donde el parámetro MA estacional Θ_1 captura el ajuste ante choques anuales. La predicción se transforma a escala de precios mediante $\hat{P}_t = \exp(\hat{\ell}_t)$.

6 Conclusiones

6.1 Síntesis metodológica

Los modelos SARIMA ofrecen un marco riguroso y parsimonioso para el análisis de series de tiempo financieras. La metodología Box-Jenkins proporciona un procedimiento sistemático: identificar la estructura mediante ACF y PACF, estimar por máxima verosimilitud, y validar mediante diagnóstico de residuos. Los criterios AIC y BIC guían la selección entre modelos competidores balanceando ajuste y complejidad.

6.2 Aplicaciones en finanzas reales

En la práctica financiera, los modelos SARIMA tienen aplicaciones directas en:

1. **Gestión de riesgo**: la predicción de volatilidad de rendimientos alimenta modelos de VaR y CVaR.
2. **Trading sistemático**: señales basadas en componentes AR/MA pueden identificar tendencias de corto plazo.
3. **Valoración de derivados**: la calibración de modelos de tasa de interés (Vasicek, CIR) se apoya en el análisis ARIMA de la serie de tasas.
4. **Planificación financiera corporativa**: pronósticos de ingresos y costos operativos con estacionalidad.
5. **Regulación y supervisión**: bancos centrales utilizan modelos SARIMA para proyectar inflación, tipo de cambio y crecimiento.

6.3 Limitaciones y extensiones

Los modelos SARIMA modelan exclusivamente la *media condicional* y asumen linealidad. Las principales limitaciones son:

- **Heterocedasticidad condicional**: los rendimientos financieros presentan *clustering* de volatilidad, que requiere extensiones GARCH.
- **No linealidad**: los modelos MS-ARIMA o de umbral (SETAR/STAR) son más apropiados para series con regímenes múltiples.
- **Alta dimensión**: para múltiples series correlacionadas, los modelos VAR o VARMA son la extensión natural.

A pesar de estas limitaciones, los modelos SARIMA permanecen como la referencia canónica en cualquier análisis de series de tiempo univariadas, por su sólida fundamentación teórica, eficiencia computacional e interpretabilidad de los coeficientes.

Referencias

1. Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.

2. Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press.
3. Tsay, R. S. (2010). *Analysis of Financial Time Series* (3rd ed.). Wiley.
4. Brockwell, P. J., & Davis, R. A. (2016). *Introduction to Time Series and Forecasting* (3rd ed.). Springer.
5. Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.
6. Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716–723.
7. Schwarz, G. (1978). Estimating the dimension of a model. *Annals of Statistics*, 6(2), 461–464.
8. Ljung, G. M., & Box, G. E. P. (1978). On a measure of lack of fit in time series models. *Biometrika*, 65(2), 297–303.
9. Dickey, D. A., & Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366), 427–431.
10. Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*.